GENBA-HYOKA - Dokumentasi Teknis Frontend

Dokumentasi ini merinci spesifikasi teknis antarmuka aplikasi Genba-Hyoka, yang dirancang untuk menjembatani komunikasi antara manajerial dan pegawai lapangan melalui platform web yang dioptimalkan untuk perangkat seluler.

BAB 1: Arsitektur & Teknologi Frontend

Bab ini menguraikan fondasi teknologi yang digunakan untuk membangun antarmuka aplikasi, memastikan sistem berjalan ringan dan responsif sesuai dengan alur kerja operasional.

1. Pendekatan Pengembangan Aplikasi:

Pendekatan pengembangan Genba-Hyoka menitikberatkan pada keseimbangan antara aksesibilitas tinggi dan fungsionalitas operasional yang kokoh. Strategi ini dibagi ke dalam beberapa poin utama:

- **Pemilihan Platform Web untuk Aksesibilitas Instan** Pemilihan website sebagai platform utama bertujuan untuk menghilangkan hambatan instalasi bagi pengguna baru. Pegawai lapangan dapat mengakses sistem secara langsung melalui browser pada perangkat masing-masing tanpa harus melalui proses distribusi aplikasi pihak ketiga. Hal ini memastikan bahwa pembaruan fitur atau perbaikan sistem dapat terkirim secara terpusat dan instan kepada seluruh pengguna secara bersamaan.
- **Strategi Desain *Mobile-First* & *Field-Oriented*** Mengingat aplikasi digunakan langsung di lokasi kejadian (Genba), antarmuka dirancang dengan prinsip *Mobile-First*. Setiap komponen visual, mulai dari formulir input hingga tombol navigasi, dioptimalkan untuk ukuran layar ponsel dan tablet. Tata letak dipastikan tetap fungsional meski diakses dalam kondisi mobilitas tinggi, memastikan arus informasi dari lapangan ke manajerial tetap lancar.
- **Optimasi Kemudahan Pengguna (UX) di Lapangan** Untuk meminimalkan kesalahan input dan kelelahan pengguna, aplikasi menerapkan standar interaksi yang sederhana:
    - **Elemen Interaktif yang Luas:** Menggunakan komponen dengan area sentuh yang besar untuk memudahkan navigasi manual.
    - **Hirarki Visual yang Jelas:** Menerapkan aturan proporsi warna **60-30-10** untuk memandu fokus pengguna pada informasi yang paling penting, seperti status tugas atau pelaporan kendala.
    - **Alur Kerja yang Ringkas:** Mengurangi jumlah langkah dalam proses pelaporan agar data lapangan dapat dikirimkan dengan secepat mungkin.
- **Kesiapan Evolusi ke Aplikasi *Native*** Pengembangan menggunakan teknologi *React Native for Web* bukan hanya untuk kebutuhan saat ini, tetapi juga sebagai langkah strategis untuk masa depan. Arsitektur kode disusun sedemikian rupa sehingga ketika

kebutuhan perusahaan berkembang untuk memerlukan fitur tingkat lanjut (seperti integrasi perangkat keras mendalam atau mode luring yang lebih kompleks), transisi menjadi aplikasi *native* (Android/iOS) dapat dilakukan dengan efisiensi waktu dan biaya yang maksimal.

- **Fasilitasi Pengguna Multi-Peran (*Multi-Role User*):** Menyadari bahwa aplikasi menargetkan perusahaan menengah ke bawah di mana satu individu sering memegang fungsi ganda (misalnya, merangkap sebagai Manajerial dan Pegawai Lapangan), sistem dirancang untuk mengakomodasi peralihan peran secara dinamis. Pengelolaan fungsi antar peran disederhanakan melalui satu antarmuka dasbor terpusat agar alur pekerjaan tetap rapi dan mudah digunakan tanpa perlu melakukan *login* berulang.

2. Ruang Lingkup Aplikasi:

Sub-bab ini merinci cakupan pekerjaan antarmuka (*frontend*) Genba-Hyoka dari hulu ke hilir serta menetapkan batasan operasional sistem agar pengembangan tetap terfokus pada tujuan utama bisnis.

A. Cakupan Fungsi Utama (End-to-End)

Pekerjaan *frontend* mencakup pengembangan logika antarmuka untuk lima klaster proses berikut:

1. **Pengelolaan Pengguna & Akses (Administrator):**
    - Pengembangan formulir pendaftaran dan pembaruan data anggota.
    - Antarmuka pengaturan profil perusahaan (identitas resmi dan logo).
    - Logika visual untuk manajemen hak akses berbasis peran (*Role-Based Access Control*).
2. **Pembuatan & Delegasi Tugas (Manajerial):**
    - Penyediaan alat bantu perancangan *template* laporan yang bersifat dinamis.
    - Antarmuka pendefinisian jenis input data (teks, angka, atau foto).
    - Sistem pemetaan tugas ke peran tertentu agar distribusi pekerjaan tepat sasaran.
3. **Pelaksanaan & Monitoring Lapangan (Pegawai & Manajerial):**
    - Ruang kerja digital bagi pegawai untuk melihat daftar tugas dan panduan pelaksanaan (SOP).
    - Pengembangan formulir input hasil kerja yang responsif.
    - *Dashboard* monitoring bagi manajerial untuk memantau progres kegiatan secara *real-time*.
4. **Evaluasi Tugas & Kinerja (Manajerial & Pegawai):**
    - Antarmuka untuk peninjauan hasil kerja, pemberian umpan balik (revisi), dan penilaian hasil.
    - Visualisasi laporan kinerja periodik yang dapat diakses oleh pegawai sebagai

bahan refleksi.

5. **Fitur Pendukung & Notifikasi:**
    - Implementasi sistem notifikasi *real-time* melalui WebSocket untuk pelaporan kendala.
    - Fungsi ekspor data laporan ke format PDF atau Excel.
    - Antarmuka pencatatan aktivitas (*audit log*) untuk transparansi sistem.

B. Batasan & Pengecualian (Exclusions)

Untuk menjaga efisiensi pengembangan tahap awal, ditetapkan batasan berikut:

- **Pemrosesan Data Berat:** *Frontend* hanya bertanggung jawab menampilkan dan memvalidasi format input. Seluruh logika kalkulasi kompleks, penyimpanan permanen, dan pengolahan basis data dilakukan sepenuhnya oleh sisi server (*backend*).
- **Mode Luring (Offline Mode):** Pada tahap ini, aplikasi diasumsikan beroperasi dengan koneksi internet aktif. Penyimpanan data saat *offline* yang bersifat masif tidak masuk dalam cakupan awal.
- **Integrasi Perangkat Keras Khusus:** Interaksi perangkat keras dibatasi pada fungsi standar *browser* (seperti kamera untuk unggah foto). Integrasi mendalam dengan sensor perangkat keras khusus lainnya dikecualikan dalam rilis ini.

3. Teknologi Frontend:

Untuk menjamin kinerja aplikasi yang responsif, aman, dan mudah dipelihara, antarmuka Genba-Hyoka dibangun di atas ekosistem **React Native** dengan dukungan **Expo for Web**. Berikut adalah rincian tumpukan teknologi (*tech stack*) yang digunakan:

- **Kerangka Kerja Utama (Core Framework)**

  React Native & Expo: Dipilih sebagai fondasi utama untuk memungkinkan pengembangan satu kode sumber (*single codebase*) yang dapat berjalan secara optimal di browser web (saat ini) dan siap didistribusikan sebagai aplikasi Android/iOS di masa mendatang.

- **Sistem Navigasi & Perutean (Routing)**

  Expo Router: Menggunakan pendekatan *file-based routing*. Teknologi ini secara otomatis memetakan struktur folder proyek menjadi URL web yang standar. Keuntungannya adalah dukungan penuh terhadap fitur browser (seperti tombol *back* dan *refresh*) serta mempermudah pemisahan area akses antara Administrator, Manajerial, dan Pegawai Lapangan.

- **Desain Visual & Antarmuka (UI/UX)**

  Tamagui: Digunakan sebagai sistem desain dan *compiler* UI. Tamagui dipilih karena kemampuannya menghasilkan gaya visual yang sangat cepat (*native-like performance*) dan fleksibel. Ini memudahkan penerapan **Aturan 60-30-10** dan skema warna **Hijau Emerald (#2ECC71)** secara konsisten di seluruh layar aplikasi.

- **Manajemen Data & State**

TanStack Query (React Query): Bertanggung jawab mengelola data dari server (REST API). Fitur unggulannya adalah manajemen *cache* otomatis dan sinkronisasi data di latar belakang, sehingga aplikasi tetap terasa cepat bagi pengguna.

Zustand: Digunakan untuk mengelola data lokal yang ringan, seperti status *login*, preferensi tampilan, dan penyimpanan token JWT secara aman di sisi klien.

- **Pengolahan Formulir & Validasi**

  React Hook Form & Zod: Menangani logika "Mesin Render Formulir Dinamis". React Hook Form memastikan performa pengisian laporan tetap ringan (tanpa *lag*), sementara Zod memastikan setiap data yang diinput (foto, teks, angka) tervalidasi dengan ketat sebelum dikirim ke server.

- **Komunikasi Real-time**

  Native WebSocket: Menggunakan protokol asli untuk jalur komunikasi instan. Ini krusial untuk fitur pelaporan kendala darurat dan notifikasi revisi tugas dari manajerial yang harus diterima pegawai lapangan dalam hitungan detik.

4. Struktur Proyek:

Untuk menjaga agar basis kode (*codebase*) tetap rapi, skalabel, dan mudah dikolaborasikan, struktur proyek disusun menggunakan arsitektur modular. Pengaturan rute memanfaatkan fitur *Route Groups* pada Expo Router untuk mendukung mekanisme Dashboard Dinamis (*Role Switcher*) bagi pengguna multi-peran.

Berikut adalah topologi direktori utama pada sistem *frontend* Genba-Hyoka:

Plaintext

```
genba-hyoka-frontend/
├── src/
│   ├── app/                # Pusat kendali Expo Router (File-based Routing)
│   │   ├── (auth)/         # Rute publik (Halaman Login)
│   │   └── (dashboard)/    # Grup rute privat dengan Layout seragam
│   │       ├── _layout.tsx # Induk Layout: Header & Role Switcher (Pojok Kanan Atas)
│   │       ├── index.tsx   # Dashboard Dinamis: Traffic Controller per Peran Aktif
│   │       ├── manager/    # Sub-rute spesifik Manajerial (contoh: pembuatan template)
│   │       └── worker/     # Sub-rute spesifik Pegawai Lapangan (contoh: input laporan)
│   │
│   ├── assets/             # Penyimpanan aset statis visual
│   │   ├── images/         # Aset gambar, logo perusahaan, dan ikon
│   │   └── fonts/          # Konfigurasi tipografi standar aplikasi
│   │
│   ├── components/         # Komponen antarmuka (UI) yang reusable (Berbasis Tamagui)
│   │   ├── forms/          # Komponen input dan pembungkus React Hook Form
│   │   ├── layouts/        # Komponen kerangka (Sidebar, Header, Bottom Tab)
│   │   └── shared/         # Komponen umum (Card, Modal, Alert)
│   │
│   ├── hooks/              # Kustomisasi React Hooks
```

```
│   │   └── queries/        # Logika pengambilan data server menggunakan TanStack Query
│   │
│   ├── store/              # Manajemen State Klien (Zustand)
│   │   └── authStore.ts    # Penyimpanan token JWT, profil, dan status peran aktif
│   │
│   ├── services/           # Modul komunikasi eksternal
│   │   ├── api/            # Konfigurasi instance REST API
│   │   └── websocket/      # Pengelolaan koneksi Native WebSocket untuk real-time
│   │
│   └── utils/              # Fungsi utilitas murni dan helper
│       ├── formatters.ts   # Pemformatan tanggal dan angka
│       └── validations.ts  # Skema validasi formulir menggunakan Zod
```

Penjelasan Modul Kunci:

- **`app/` (Sistem Navigasi & Dasbor Dinamis):** Berfungsi sebagai tulang punggung navigasi. Alih-alih memisahkan dasbor secara kaku, file `_layout.tsx` akan memuat komponen *Role Switcher*, sementara `index.tsx` merender konten secara reaktif berdasarkan peran yang sedang aktif. Wewenang spesifik diisolasi secara eksklusif di dalam direktorinya masing-masing: `admin/` untuk manajemen sistem dan profil pengguna, `manager/` untuk fungsi seperti perancangan *template* laporan, dan `worker/` untuk tugas operasional. Pemisahan ini mencegah akses silang dari pengguna di luar level wewenang yang diizinkan.
- **`assets/` (Aset Visual):** Direktori untuk elemen visual statis. Aset desain untuk logo perusahaan dipastikan menggunakan format dengan latar belakang warna solid (tersedia dalam tiga versi: ikon, bertumpuk, dan menyamping) untuk memastikan harmoni visual dengan tata letak aplikasi dan menghindari tabrakan kontras.
- **`components/` (Sistem Desain):** Seluruh komponen UI dibangun dengan Tamagui untuk menjamin performa maksimal. Desain komponen secara ketat mematuhi pedoman proporsi warna 60-30-10.
- **`store/` & `hooks/` (Manajemen Data):** Zustand menangani *state* lokal seperti peralihan peran (multi-peran), sedangkan TanStack Query dikhususkan untuk mengelola lalu lintas data dari server.
- **`utils/` (Validasi Data):** Setiap bentuk masukan formulir dari komponen UI akan divalidasi secara terpusat menggunakan skema Zod yang berada di dalam direktori ini sebelum dikirimkan melalui layanan API.

BAB 2: Manajemen Akses Berbasis Peran (RBAC)

Bab ini menguraikan arsitektur keamanan dan mekanisme pengontrolan akses di sisi klien, memastikan antarmuka secara presisi menampilkan fitur yang sesuai dengan wewenang (termasuk *multi-role*) yang diberikan oleh server.

2.1 Autentikasi & Manajemen Sesi (Dual-Token Strategy)

Aplikasi mengadopsi mekanisme sesi berbasis *Stateless JWT* dengan strategi dua lapis token untuk menjaga keamanan tanpa mengganggu mobilitas pengguna.

- **Access Token:** Berlaku singkat (1 jam), dikirimkan dalam *header* otorisasi (`Bearer <token>`) pada setiap permintaan REST API.
- **Refresh Token:** Berlaku lebih lama (9 jam), digunakan untuk memperbarui Access Token secara otomatis melalui mekanisme *silent refresh* di latar belakang tanpa mengharuskan pengguna *login* ulang selama sesi masih aktif.
- **Keamanan Klien:** Token disimpan menggunakan media penyimpanan aman (seperti *SecureStore* atau *encrypted storage*) untuk mencegah akses ilegal dari skrip pihak ketiga.

2.2 Otorisasi & Interogasi *Bitmask*

Untuk mendukung fleksibilitas pengguna *multi-role*, *frontend* mengonsumsi data otorisasi melalui objek `prm` (Permissions) yang dikirimkan oleh server dalam *payload* JWT.

- **Struktur Data:** Hak akses untuk setiap modul (seperti `tsk` untuk Tugas atau `rpt` untuk Laporan) direpresentasikan dalam nilai bilangan bulat (integer).
- **Operasi di Sisi Klien:** Pengembang *frontend* wajib menggunakan operasi *Bitwise AND* (`&`) untuk memvalidasi hak akses sebelum merender komponen UI tertentu. Contoh: Untuk menampilkan tombol "Hapus Laporan", sistem akan memeriksa apakah `(permissions.rpt & 8) > 0` (di mana 8 mewakili bit untuk akses `DELETE`).
- **Reaktivitas Peran:** Setiap kali pengguna beralih peran melalui *Role Switcher*, aplikasi akan memvalidasi ulang hak akses aktif terhadap skema *bitmask* terbaru.

2.3 Proteksi Perutean (*Route Guards*)

Pengamanan jalur navigasi pada Expo Router diatur secara sistematis untuk mencegah akses

ke area yang tidak sah:

- **Middleware Otorisasi:** Setiap perpindahan rute menuju folder privat (seperti `/admin` atau `/manager`) akan dicegat oleh *middleware* untuk memverifikasi apakah `activeRole` pengguna memiliki izin yang memadai.
- **Pengalihan Otomatis:** Jika verifikasi gagal, pengguna akan dialihkan kembali ke dasbor utama atau halaman *login* dengan pemberitahuan akses ditolak.

2.4 Keamanan Komunikasi *Real-time*

Sesuai standar keamanan *backend*, aplikasi tidak mengirimkan token JWT secara terbuka melalui URL saat menginisialisasi koneksi WebSocket.

- **Proses *Handshake*:** *Frontend* akan meminta *One-Time Ticket* (OTT) melalui jalur REST API yang aman sebelum membuka koneksi.
- **Validasi Tiket:** Tiket sekali pakai tersebut kemudian dikirimkan saat proses jabat tangan (*handshake*) WebSocket untuk memverifikasi identitas pengguna secara instan.

BAB 3: Komunikasi Data & Interaksi Sistem

3.1 Pengelolaan REST API

REST API berfungsi sebagai jalur komunikasi utama untuk seluruh transaksi data yang bersifat *request-response* di dalam sistem Genba-Hyoka. Implementasi ini memastikan pertukaran informasi terstruktur secara konsisten antara *frontend* (React Native) dan *backend* (Elysia.js).

A. Standar Protokol & Format Data

- **Protokol:** Seluruh komunikasi dilakukan melalui protokol HTTPS untuk menjamin enkripsi data saat transit.
- **Format Data:** Menggunakan JSON (*JavaScript Object Notation*) sebagai standar tunggal untuk pengiriman *payload* dari klien maupun penerimaan respons dari server.
- **Basis URL:** Setiap permintaan diarahkan pada titik akhir (*endpoint*) dengan penomoran versi (misal: `/v1/...`) untuk menjamin stabilitas aplikasi jika terjadi pembaruan struktur API di masa mendatang.

B. Penggunaan Metode HTTP

Aplikasi menggunakan metode HTTP standar untuk merepresentasikan tindakan terhadap sumber data:

- **GET:** Digunakan untuk mengambil data, seperti daftar tugas operasional atau detail

profil pengguna.

- **POST:** Digunakan untuk membuat entitas baru, seperti pengiriman laporan lapangan awal atau pembuatan tiket WebSocket.
- **PATCH / PUT:** Digunakan untuk memperbarui data yang sudah ada, misalnya melakukan revisi pada laporan atau mengubah status tugas.
- **DELETE:** Digunakan untuk menghapus data tertentu (terbatas pada peran Administrator).

C. Otorisasi Header

Setiap permintaan ke rute yang dilindungi wajib menyertakan identitas pengguna dalam *header* permintaan.

- **Format:** `Authorization: Bearer <access_token>`
- **Keamanan:** Jika *header* tidak disertakan atau token tidak valid, server akan memberikan respons status `401 Unauthorized`, yang kemudian akan diproses oleh *frontend* melalui mekanisme *silent refresh*.

D. Standar Format Respons

Untuk memudahkan proses *parsing* data di sisi klien, server memberikan struktur respons yang seragam:

1. **Respons Sukses:**

   JSON

   ```
   {
     "success": true,
     "message": "Pesan deskriptif keberhasilan",
     "data": { ... } // Objek atau array data yang diminta
   }
   ```

2. **Respons Gagal:**

   JSON

   ```
   {
     "success": false,
     "message": "Pesan alasan kegagalan",
     "error_code": "KODE_ERROR_UNIK",
     "details": [] // Informasi tambahan mengenai error (jika ada)
   }
   ```

3.2 Komunikasi Real-time (WebSocket)

Fitur komunikasi *real-time* sangat krusial bagi operasional Genba-Hyoka untuk memastikan aliran informasi antara lapangan dan manajerial terjadi secara instan tanpa perlu memuat ulang halaman (*refresh*).

A. Tujuan Penggunaan

Jalur WebSocket digunakan secara khusus untuk menangani fungsi-fungsi berikut:

- **Notifikasi Instan:** Pemberitahuan tugas baru atau revisi laporan yang dikirim oleh manajerial.
- **Pembaruan Status:** Perubahan status tugas secara langsung pada dasbor pemantauan.
- **Pelaporan Kendala:** Pengiriman sinyal urgensi dari lapangan ke pihak kantor secara *real-time*.

B. Mekanisme Jabat Tangan Keamanan (Secure Handshake)

Sesuai dengan protokol keamanan *backend*, aplikasi tidak mengirimkan token JWT melalui parameter URL. Sebagai gantinya, digunakan mekanisme *One-Time Ticket* (OTT):

1. **Request Ticket:** *Frontend* melakukan permintaan ke *endpoint* `POST /v1/auth/ws-ticket` dengan menyertakan JWT pada *header*.
2. **Ticket Issued:** Server mengembalikan `ticket_id` yang berlaku selama 60 detik.
3. **Connection:** *Frontend* membuka koneksi ke alamat `wss://api.genba.com/ws?ticket={ticket_id}`.

C. Mekanisme Heartbeat (Ping-Pong)

Untuk mendeteksi koneksi "zombi" atau kegagalan jalur yang tidak terdeteksi oleh lapisan transport, sistem menerapkan protokol *heartbeat*:

- **Aksi:** *Frontend* secara berkala mengirimkan paket kecil (*Ping*) ke server dalam interval waktu tertentu (misal: 30 detik).
- **Validasi:** Jika server tidak memberikan respons (*Pong*) dalam batas waktu yang ditentukan, *frontend* akan menganggap koneksi telah mati, memutus jalur secara paksa (`CLOSED`), dan memulai prosedur koneksi ulang.

D. Struktur Payload Pesan

Untuk memudahkan proses *parsing* dan penanganan data, setiap pesan yang diterima atau dikirim menggunakan format JSON yang seragam:

JSON

```
{
  "event": "string",     // Contoh: "NEW_TASK", "STATUS_UPDATED",
"REVISION_REQUIRED"
  "data": { ... },       // Objek data terkait event tersebut
  "timestamp": "iso8601" // Waktu pengiriman data dari server
}
```

E. Strategi Reconnection & Sinkronisasi

Mengingat kondisi jaringan di lapangan (Genba) sering kali tidak stabil, *frontend* mengimplementasikan logika ketahanan koneksi sebagai berikut:

- **Exponential Backoff:** Jika koneksi terputus, aplikasi akan melakukan percobaan ulang secara otomatis dengan jeda yang meningkat secara eksponensial (mulai dari 1 detik, lalu 2s, 4s, 8s, hingga maksimal 30 detik).
- **Auto-Sync on Connect:** Setiap kali status koneksi kembali menjadi `CONNECTED`,

aplikasi secara otomatis akan memicu fungsi sinkronisasi (*Sync*) untuk mengambil data terbaru yang mungkin terlewat selama periode luring (*offline*). Hal ini menjamin tidak ada instruksi kerja atau laporan yang tertinggal.

3.3 Mekanisme Interceptor & Otorisasi Token

Untuk mengelola *Dual-Token Strategy* (Access Token & Refresh Token) dengan aman dan efisien, aplikasi memanfaatkan fitur **Interceptor** pada pustaka HTTP *Client* (seperti Axios atau *custom fetch wrapper*). Interceptor ini bekerja di latar belakang untuk mencegat (*intercept*) setiap permintaan yang keluar dan respons yang masuk.

A. Injeksi Otorisasi Otomatis (Request Interceptor)

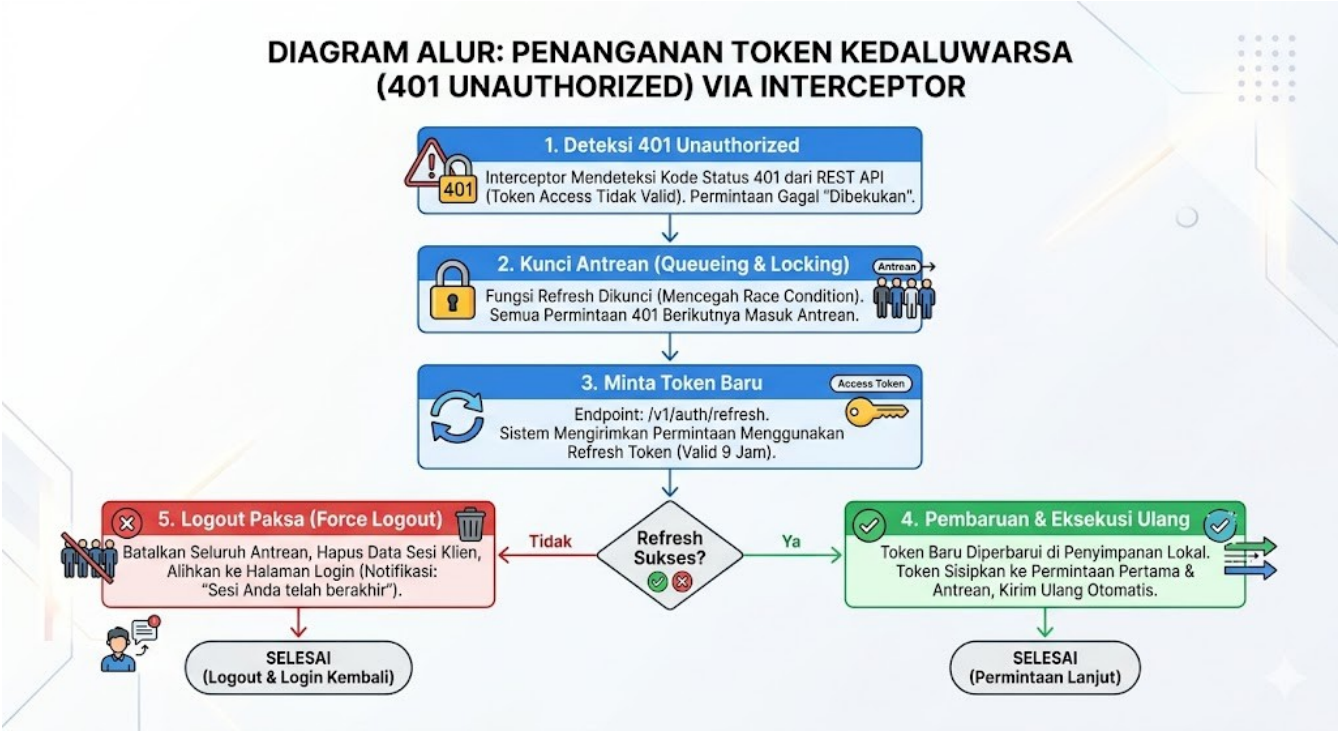
Alih-alih menyisipkan token secara manual di setiap fungsi pengambilan data, *Request Interceptor* bertugas secara terpusat:

- Sebelum permintaan dikirim ke server, sistem membaca nilai *Access Token* dari *state* lokal (Zustand) atau penyimpanan aman (*SecureStore*).
- Jika token tersedia dan rute yang dituju bukan rute publik (seperti `/login`), interceptor secara otomatis menyisipkan *header* `Authorization: Bearer <access_token>` ke dalam *payload* permintaan.

B. Mekanisme *Silent Refresh* (Response Interceptor)

Mengingat *Access Token* memiliki umur pendek (1 jam) demi keamanan, *Response Interceptor* bertugas menangani momen ketika token tersebut kedaluwarsa, tanpa mengganggu interaksi pengguna di layar.

Alur pemulihan (*silent refresh*) dieksekusi dengan urutan yang ketat untuk mencegah *error* berantai:



3.4 Penanganan Kesalahan & Kendala Jaringan (Error Handling)

Kondisi operasional di lokasi lapangan (*Genba*) sangat rentan terhadap fluktuasi sinyal internet. Oleh karena itu, aplikasi harus mampu membedakan antara masalah server, masalah validasi data, dan masalah jaringan proksimal, serta memberikan umpan balik (*feedback*) yang transparan kepada pengguna.

A. Penanganan Kesalahan API Terpusat

Aplikasi memanfaatkan standarisasi respons *error* dari server (melalui struktur `success: false` dan `error_code`) untuk memberikan pesan yang relevan.

- **Pencegatan Terpusat:** *Response Interceptor* membaca kode status HTTP. Kesalahan

umum (seperti `400 Bad Request` atau `422 Unprocessable Entity`) akan diekstrak properti `message` dan `details`-nya.

- **Translasi Pesan (UI):** Pesan kesalahan teknis dari *backend* tidak langsung ditampilkan mentah-mentah. *Frontend* menggunakan modul *Toast* atau *Snackbar Alert* untuk menampilkan terjemahan visual yang ramah pengguna (misal: "Gagal menyimpan laporan: Foto lokasi belum diunggah").

B. Resiliensi Terhadap Kendala Jaringan

Ketika permintaan data gagal karena *timeout* atau koneksi terputus murni (bukan karena ditolak server), aplikasi mengimplementasikan strategi berikut:

- **Mekanisme *Retry* Otomatis:** Menggunakan kapabilitas TanStack Query, permintaan pengambilan data (GET) yang gagal karena *network error* akan diulang secara otomatis hingga 3 kali dengan jeda waktu (*backoff*) sebelum akhirnya menyerah dan menampilkan pesan *error*.
- **Status "Offline" Visual:** Jika detektor perangkat mengenali bahwa koneksi internet seluler atau WiFi benar-benar mati, aplikasi akan menampilkan spanduk (*banner*) persisten berwarna peringatan di bagian atas layar: *"Anda sedang di luar jangkauan jaringan. Menunggu koneksi..."*

C. Perlindungan Batas Kesalahan (*Error Boundaries*)

Untuk mencegah aplikasi mengalami *crash* total (layar putih atau aplikasi tertutup paksa) akibat kesalahan tak terduga (*runtime exception*) pada kode *frontend*:

- **Global Error Boundary:** Sebuah komponen pembungkus tingkat tertinggi akan menangkap kesalahan *rendering*.
- **Fallback UI:** Jika komponen tertentu gagal dimuat, aplikasi akan menggantinya dengan tampilan aman (Fallback UI) yang berisi pesan *"Terjadi kesalahan saat memuat halaman ini"* dilengkapi tombol *"Muat Ulang Halaman"*. Hal ini memastikan pengguna masih bisa menavigasi ke menu lain alih-alih harus memulai ulang aplikasi dari awal.

D. Penanganan Formulir Idempoten

Untuk mencegah pengiriman data ganda ketika pengguna menekan tombol "Kirim" berkali-kali saat sinyal lambat (lag):

- **Penguncian Tombol (*Button State Locking*):** Setiap tombol aksi utama akan berubah menjadi *loading state* (non-aktif) segera setelah diklik, dan baru akan kembali aktif jika server telah mengembalikan respons (sukses atau gagal) atau batas waktu *timeout* tercapai.

BAB 4: Fokus Komponen Antarmuka (UI) Utama & Panduan Visual

Bab ini merinci standar estetika dan panduan visual yang diterapkan pada antarmuka Genba-Hyoka untuk memastikan konsistensi, keterbacaan, dan kenyamanan pengguna dalam berbagai kondisi pencahayaan di lapangan.

4.1 Skema Warna (Color Palette)

Sistem desain menggunakan kombinasi warna yang kontras untuk memisahkan antara elemen struktural dan elemen aksi.

Kategori	Kode Hex	Penggunaan
Primary	#2ECC71	Tombol aksi utama, penanda sukses, dan elemen perhatian.
Secondary	#2C3E50	Navigasi, ikon, dan struktur utama.
Background	#FFFFFF / #F8F9FA	Area konten dan permukaan kartu informasi.
Text Muted	#95A5A6	Teks placeholder, tanggal, dan informasi sekunder.

4.2 Proporsi Penggunaan (Aturan 60-30-10)

Agar desain tetap seimbang dan profesional, diterapkan rasio penggunaan warna sebagai berikut:

- **60% Warna Dasar (Putih / Abu-abu Muda):** Digunakan sebagai ruang kosong (*whitespace*) untuk menjaga kejelasan konten dan mencegah kelelahan mata saat menatap layar dalam waktu lama.
- **30% Warna Sekunder (Arang Gelap):** Digunakan untuk teks utama, ikon, dan latar belakang navigasi agar informasi dan hierarki sistem terbaca dengan sangat jelas.
- **10% Warna Primary (Hijau Emerald):** Digunakan secara eksklusif untuk elemen *Call to Action* (CTA), tombol kirim, atau indikator status positif yang memerlukan perhatian atau tindakan cepat dari pengguna.

4.3 Komponen Ikonografi & Aset Visual

Sesuai dengan kebutuhan dokumentasi teknis, seluruh aset visual mengikuti standar berikut:

- **Format Logo:** Desain logo disediakan dalam tiga versi: *icon-only*, *stacked*, dan *side version*.
- **Gaya Visual:** Menggunakan latar belakang solid (bukan transparan) untuk memastikan visibilitas yang konsisten di atas berbagai elemen UI.

- **Ikon Navigasi:** Menggunakan gaya minimalis dengan warna *Secondary* untuk kondisi tidak aktif, dan warna *Primary* untuk kondisi aktif.

4.3 Fitur Utama UI

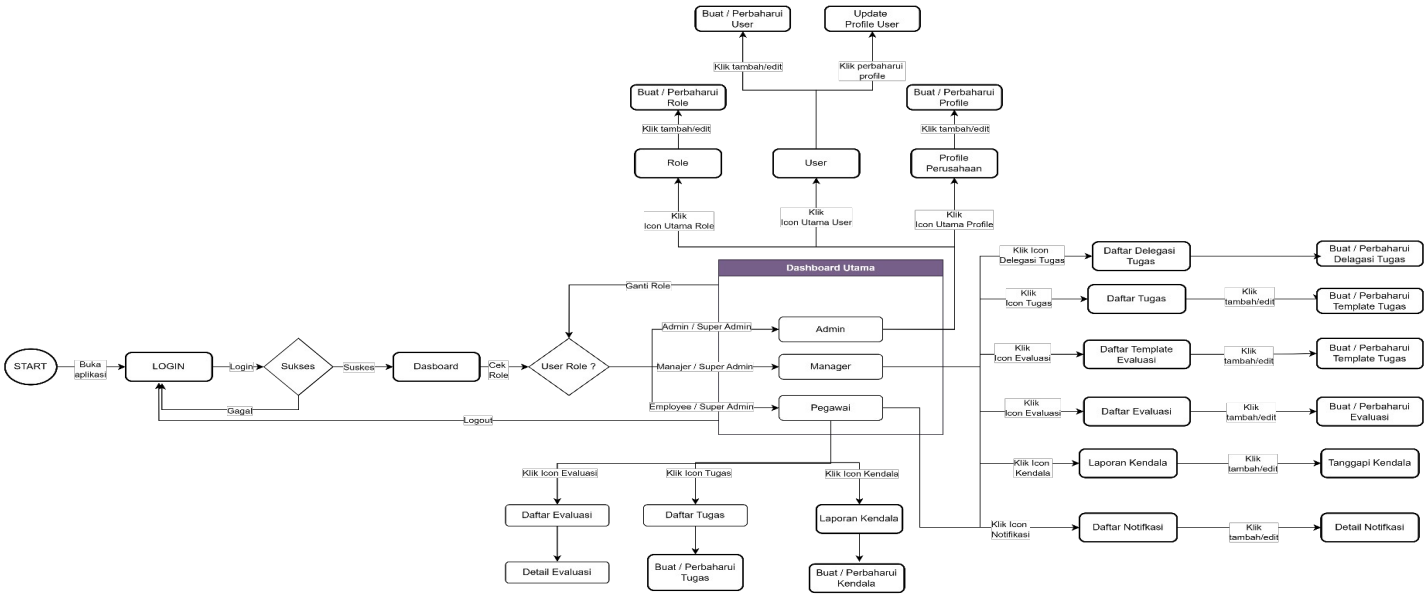


Figure 1: Diagram Pergerakan UI Genba-Hyoka

Diagram *User Flow* di atas memetakan perjalanan interaksi pengguna di dalam aplikasi Genba-Hyoka, mulai dari proses autentikasi hingga penyelesaian tugas operasional berdasarkan peran (*Role-Based Access*). Alur ini mengadopsi model *Hub and Spoke* tersentralisasi, di mana **Dashboard Utama** bertindak sebagai pusat navigasi.

Berikut adalah rincian pergerakan antarmuka berdasarkan area fungsionalnya:

1. Fase Autentikasi & Gerbang Logika (Entry Flow)

- **Login & Validasi:** Pengguna memulai dengan membuka aplikasi dan masuk ke layar LOGIN. Sistem akan memvalidasi kredensial. Jika gagal, pengguna dikembalikan ke layar login. Jika sukses, pengguna diarahkan menuju proses inisialisasi Dasbor.
- **Pengecekan Peran (Role Routing):** Sebelum merender layar, sistem melewati gerbang logika User Role? untuk membaca otorisasi *Bitmask* pengguna.
- **Role Switcher:** Jika pengguna memiliki lebih dari satu jabatan (Multi-Role), mereka dapat menavigasi kembali ke gerbang logika ini kapan saja melalui fitur Ganti Role tanpa perlu *logout*, yang akan memperbarui antarmuka Dasbor Utama secara instan.

2. Alur Administrator (Manajemen Sistem)

Area ini diakses secara eksklusif oleh Admin/Super Admin untuk mengatur struktur organisasi. Dari Dasbor Utama, Admin dapat menavigasi ke tiga modul utama:

- **Manajemen Role:** Klik ikon *Utama Role* > melihat daftar Role > klik *tambah/edit* untuk masuk ke layar formulir Buat/Perbarui Role.
- **Manajemen User:** Klik ikon *Utama User* > melihat daftar User > masuk ke layar Buat/Perbarui User dan melengkapi informasi user.
- **Profil Perusahaan:** Klik ikon *Utama Profile* > mengelola identitas perusahaan melalui layar Buat/Perbarui Profile.

3. Alur Manajerial (Pengendalian & Pemantauan)

Area fungsional dengan kepadatan fitur tertinggi, dirancang untuk Manajer agar dapat mengendalikan operasi lapangan:

- **Delegasi & Template:** Manajer mengelola arsitektur tugas melalui rute Daftar Delegasi Tugas dan perancangan formulir dinamis di Daftar Template Evaluasi.
- **Tugas & Evaluasi:** Melalui Dasbor, Manajer dapat memantau Daftar Tugas yang sedang berjalan, serta meninjau hasil kerja pegawai melalui menu Daftar Evaluasi untuk memberikan penilaian atau revisi.
- **Respon Cepat:** Manajer dilengkapi dengan akses ke Laporan Kendala untuk merespons masalah darurat (melalui layar Tanggapi Kendala) dan memantau pembaruan sistem via Daftar Notifikasi.

4. Alur Pegawai (Operasional Lapangan - Genba)

Area ini dirancang sangat ringkas (*compact*) agar pekerja lapangan dapat fokus pada eksekusi tanpa terdistraksi menu yang rumit:

- **Eksekusi SOP:** Dari Dasbor, pegawai membuka Daftar Tugas dan langsung masuk ke layar pengisian form dinamis (Buat/Perbarui Tugas) untuk mengirimkan laporan pekerjaan.
- **Umpan Balik:** Pegawai dapat meninjau hasil kerja mereka yang telah dinilai Manajer melalui rute Daftar Evaluasi > Detail Evaluasi.
- **Pelaporan Isu:** Jika terjadi masalah di lapangan, pegawai menekan ikon *Kendala* untuk masuk ke layar Buat/Perbarui Kendala, yang akan mengirimkan sinyal urgensi langsung ke Dasbor Manajer secara *real-time*.